SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Oral Defense week 10

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY –2026

Contents

Oral Defense week 10 3

1. (Extension - Task C.7) Describe your Sentiment Analysis implementation. How do you convert textual news/social media data into a numerical "Sentiment Score"? 3

2. Explain the "Classification" model you built for the extension. Why is it sometimes easier to predict "Up/Down" (Binary) than the exact "Price" (Regression)? 3

3.(Lecture 9 Concept): Algorithmic trading is a classic use case for Reinforcement Learning (RL). If you were to transition your project from simply predicting prices to trading autonomously using RL, how would you define the "State", the "Action Space", and the "Reward"? 4

Oral Defense week 10

1. (Extension- Task C.7) Describe your Sentiment Analysis implementation. How do you convert textual news/social media data into a numerical "Sentiment Score"?

In my C.7 extension, I first collected textual news data from FinViz and Alpha Vantage, then standardized the columns so both sources had the same structure, such as date, headline, source, and url. After that, I filtered and merged the news and aligned it with the stock date range before doing sentiment analysis.

To convert text into a numerical sentiment score, I used two methods. The first was VADER, where each headline is passed into a sentiment analyzer and gets a vader_compound score. Then I aggregate those headline-level scores into daily features: the daily mean, daily sum, and daily count. So instead of raw text, the model receives numbers such as sent_mean, sent_sum, and sent_count.

The second method was FinBERT, which is a finance-specific language model. For each headline, it predicts probabilities for negative, neutral, and positive sentiment. I then convert those probabilities into one continuous score using finbert_score = P(positive) - P(negative). After that, I aggregate these scores by day into finbert_mean, finbert_sum, and finbert_count. This was my independent research component, because it goes beyond a simple lexicon-based method.

So the key idea is: raw text is not directly used by the classifier. I first transform text into numbers that represent sentiment intensity, and then merge those numbers with stock features.

2. Explain the "Classification" model you built for the extension. Why is it sometimes easier to predict "Up/Down" (Binary) than the exact "Price" (Regression)?

For the extension, I built a binary classification model. The target label is called label_up, and it is created by comparing the next day's closing price with the current day's closing price. If the next day closes higher, the label is 1; otherwise it is 0. This directly turns the problem into predicting UP or DOWN instead of predicting the exact future price.

My input features included both price-based features and sentiment-based features. The price features were ret_1d, ma_5, ma_10, vol_5, and volchg_1d. Then I trained classifiers including Logistic Regression and Random Forest, and compared price-only models with price-plus-sentiment models.

It is often easier to predict Up/Down than the exact Price because classification asks a simpler question. In regression, the model has to predict an exact numerical value, and even a small market shock can make the predicted price noticeably wrong. In classification, the model only needs to capture the direction of movement. So even if it cannot estimate the exact closing price, it may still correctly identify whether the market goes up or down. That reduces the complexity of the target and often makes the learning problem more stable.

In my results, this classification setup produced moderate but meaningful performance. For example, RandomForest (Price + VADER) achieved Accuracy 0.5645 and F1 0.6197, while LogReg (Price-only) achieved F1 0.6216. This shows the classifier was able to learn some directional signal even though it was not perfect.

I evaluated the models with accuracy, precision, recall, F1-score, and confusion matrix, because for classification it is important to understand not only how often the model is correct, but also what kind of mistakes it makes.

3.(Lecture 9 Concept): Algorithmic trading is a classic use case for Reinforcement Learning (RL). If you were to transition your project from simply predicting prices to trading autonomously using RL, how would you define the "State", the "Action Space", and the "Reward"?

If I changed my project from prediction to autonomous trading with Reinforcement Learning, I would model it as an agent interacting with the stock market environment over time. In the lecture, RL is about making a good sequence of decisions under uncertainty and maximising cumulative reward, not just immediate reward.

State:
The state should contain the information the agent needs to choose the next action. For my project, that would include recent price features like return, moving averages, volatility, and volume change, plus daily sentiment features from VADER and FinBERT, and also portfolio information such as cash, shares held, and current position. This matches the lecture idea that the agent state is its internal representation of the environment.

Action Space:
The simplest action space would be Buy, Hold, Sell. If I wanted a more realistic system later, I could extend it to different trade sizes such as buy 25%, buy 50%, sell part, or sell all. The lecture describes a policy as a mapping from each state to an action, so these would be the actions available to the policy.

Reward:
The reward should reflect how good the trading decision was. A good choice would be the change in portfolio value after each action. A better reward would subtract transaction costs and maybe penalise excessive risk or overtrading, because RL should optimise long-term return, not just one-step profit.

In one sentence:
State = market features + sentiment features + portfolio status; Action = Buy/Hold/Sell; Reward = profit in portfolio value, adjusted for cost and risk.